

尚硅谷嵌入式技术之四轴无人机_遥控器

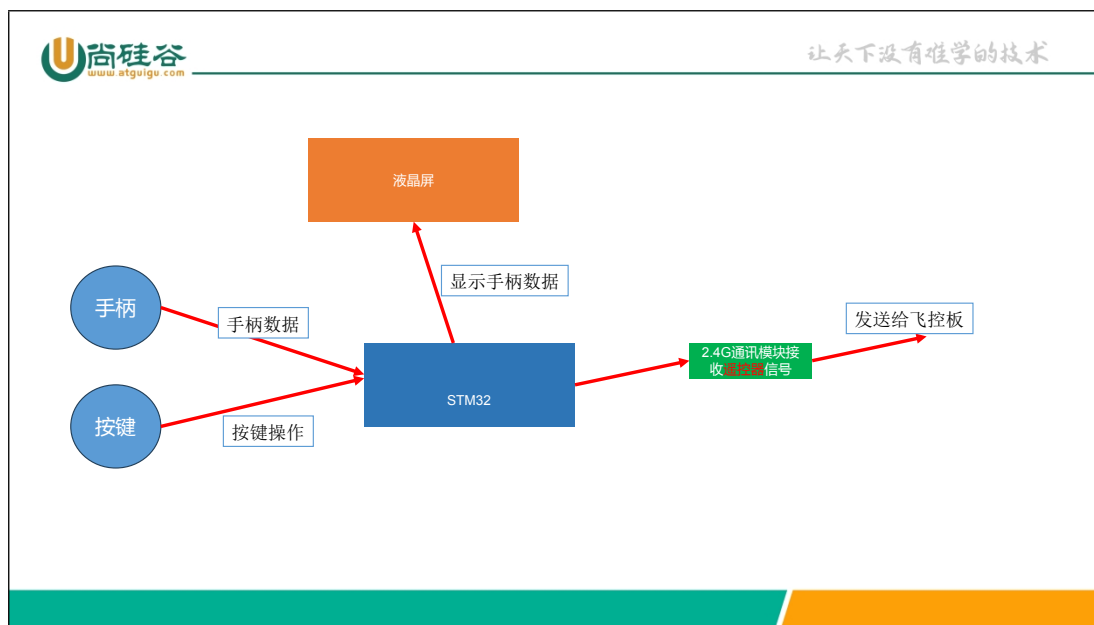
(作者：尚硅谷研究院)

版本：V1.1.0

第 1 章 项目需求

本遥控器是为了对无人机实现远程操控而设计

第 2 章 系统总体设计



第 3 章 公共层

3.1 Common_Config.h

```
#ifndef __COMMON_CONFIG_H
#define __COMMON_CONFIG_H

/* 0. 定义通讯的帧头 */
#define FRAME_0 0xAA
#define FRAME_1 0xAF
#define FRAME_2 0x03

/* 1. 通用状态枚举类型 */
```

```
typedef enum
{
    Common_OK = 0,
    Common_ERROR,
    Common_OTHER
} Common_Status;

/* 2. 遥感和按键数据封装结构体 */
typedef struct
{
    uint16_t THR; /* 油门 Throttle */
    uint16_t YAW; /* 横滚 */
    uint16_t ROL; /* 横滚 */
    uint16_t PIT; /* 俯仰 */

    uint8_t isFixHeightPoint; /* 是否定高定点 */
    uint8_t isPowerDown; /* 是否关机 */
} JoyStick_Struct;

typedef struct
{
    int16_t THRBias; /* 油门 偏差 */
    int16_t YAWBias; /* 横滚 偏差 */
    int16_t ROLBias; /* 横滚 偏差 */
    int16_t PITBias; /* 俯仰 偏差 */
} JoyStickBias_Struct;

#endif
```

第 4 章 电源模块

参考飞控板.

第 5 章 手柄和按键数据处理模块

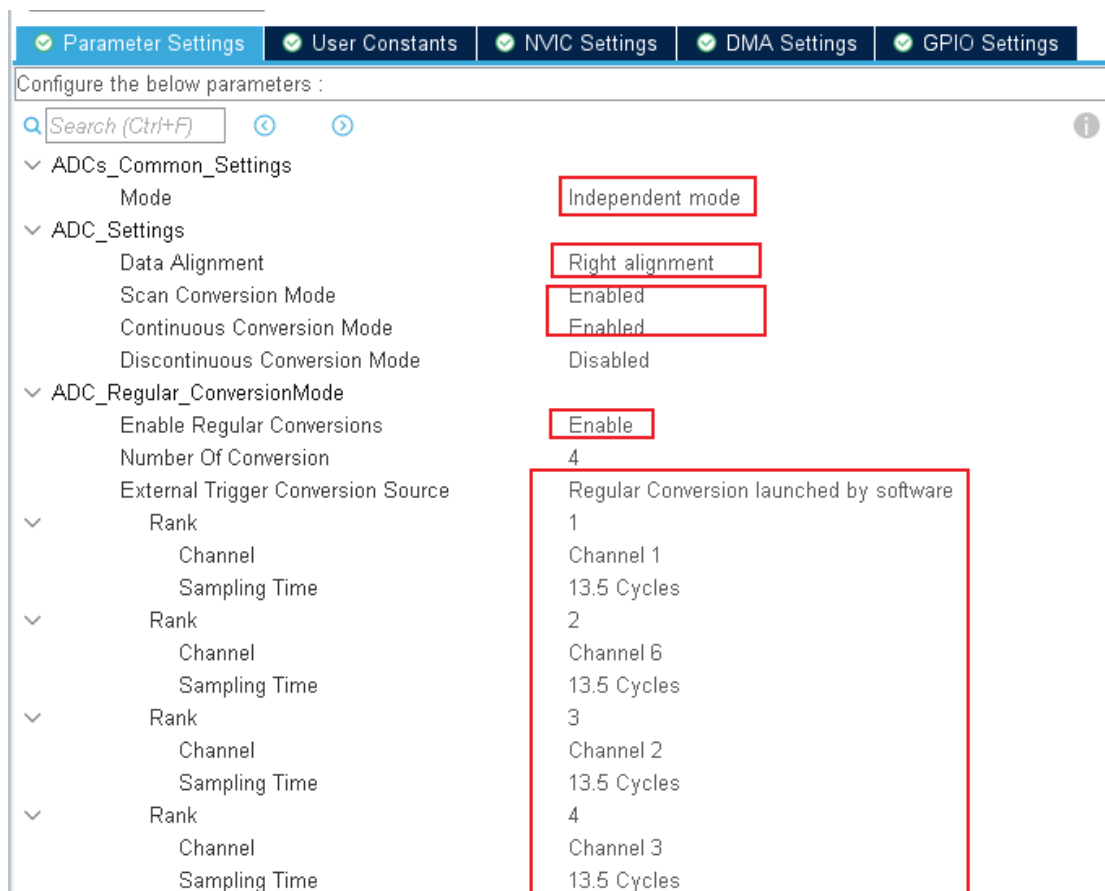
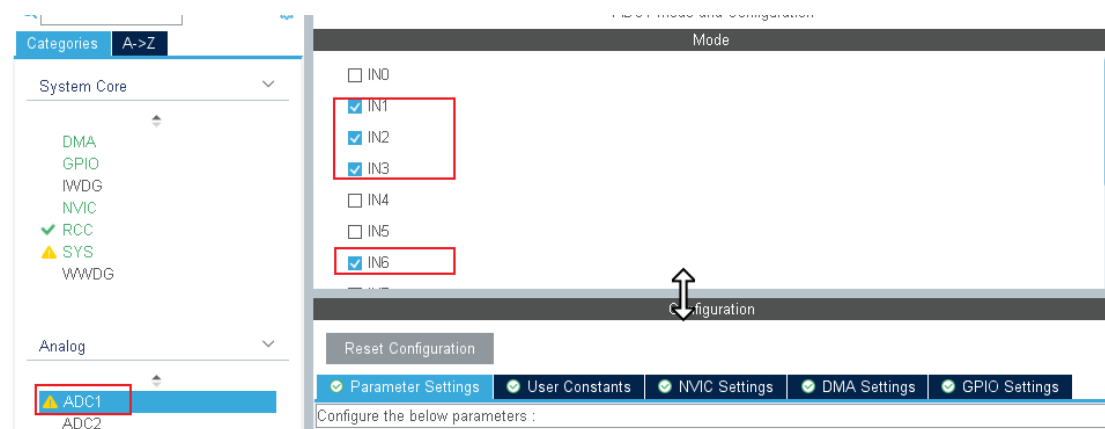
5.1 STM32CubeMX 配置

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

5.1.1 按键配置

Pin Name	Signal on Pin	GPIO output...	GPIO mode	GPIO Pull-u...	Ma...	User Label	Modified
PB14	n/a	n/a	Input mode	Pull-up	n/a	KEY_DOWN	☒
PB13	n/a	n/a	Input mode	Pull-up	n/a	KEY_LEFT	☒
PB2	n/a	n/a	Input mode	Pull-up	n/a	KEY_LEFT_UP	☒
PB12	n/a	n/a	Input mode	Pull-up	n/a	KEY_RIGHT	☒
PB10	n/a	n/a	Input mode	Pull-up	n/a	KEY_RIGHT_UP	☒
PB11	n/a	n/a	Input mode	Pull-up	n/a	KEY_UP	☒

5.1.2 手柄配置



5.2 硬件接口层

5.2.1 Inf_JoyStickKey.h

```
#ifndef __INF_JOYSTICKKEY_H
#define __INF_JOYSTICKKEY_H

#include "Common_Debug.h"
#include "Common_Config.h"
#include "Common_Delay.h"
#include "main.h"
#include "adc.h"

#define READ_KEY_LEFT (HAL_GPIO_ReadPin(KEY_LEFT_GPIO_Port, KEY_LEFT_Pin))
#define READ_KEY_RIGHT (HAL_GPIO_ReadPin(KEY_RIGHT_GPIO_Port, KEY_RIGHT_Pin))
#define READ_KEY_UP (HAL_GPIO_ReadPin(KEY_UP_GPIO_Port, KEY_UP_Pin))
#define READ_KEY_DOWN (HAL_GPIO_ReadPin(KEY_DOWN_GPIO_Port, KEY_DOWN_Pin))

#define READ_KEY_LEFT_UP (HAL_GPIO_ReadPin(KEY_LEFT_UP_GPIO_Port, KEY_LEFT_UP_Pin))
#define READ_KEY_RIGHT_UP (HAL_GPIO_ReadPin(KEY_RIGHT_UP_GPIO_Port,
KEY_RIGHT_UP_Pin))

#define NOKEY_PRESS 0
#define KEY_LEFT_PRESS 1
#define KEY_RIGHT_PRESS 2
#define KEY_UP_PRESS 3
#define KEY_DOWN_PRESS 4
#define KEY_LEFT_UP_PRESS 5
#define KEY_RIGHT_UP_PRESS 6
#define KEY_LEFT_UP_LONG_PRESS 7
#define KEY_RIGHT_UP_LONG_PRESS 8

extern JoyStick_Struct joyStick;

void Inf_JoyStickKey_Init(void);
void Inf_JoyStickKey_JoyStickScan(void);
uint8_t Inf_JoyStickKey_KeyScan(void);
```

```
#endif
```

5.2.2 Inf_JoyStickKey.c

```
#include "Inf_JoyStickKey.h"

/* 存储摇杆数据：
    0: left 杆-> 左右 -> 偏航
    1: left 杆-> 前后 -> 油门
    2: right 杆-> 左右 -> 横滚
    3: right 杆-> 前后 -> 俯仰
*/
uint16_t      joyStickBuff[4];
JoyStick_Struct joyStick;

/**
 * @description: 摇杆和按键初始化
 * @return {*}
 */
void Inf_JoyStickKey_Init(void)
{
    /* 启动 ADC 采集，因为有 4 个通道，所以要开启 DMA */
    debug_printfln("摇杆和按键模块：开始启动");
    HAL_ADCEx_Calibration_Start(&hadc1);
    HAL_ADC_Start_DMA(&hadc1, (uint32_t *)joyStickBuff, 4);
    debug_printfln("摇杆和按键模块：启动完成");
}

/**
 * @description: 扫描遥感数据
 * @return {*}
 */
void Inf_JoyStickKey_JoyStickScan(void)
{
    joyStick.THR = joyStickBuff[0];
    joyStick.YAW = joyStickBuff[1];
    joyStick.PIT = joyStickBuff[2];
}
```

```
joyStick.ROL = joyStickBuff[3];
}

/**
 * @description: 按键扫描，检测按键是否被按下
 * @return {*}
 */
uint8_t Inf_JoyStickKey_KeyScan(void)
{
    /* 表示有按键按下 */
    if(READ_KEY_LEFT == 0 || READ_KEY_RIGHT == 0 ||
        READ_KEY_UP == 0 || READ_KEY_DOWN == 0 ||
        READ_KEY_LEFT_UP == 0 || READ_KEY_RIGHT_UP == 0)
    {
        common_delay_ms(50);
        if(READ_KEY_LEFT == 0)
        {
            /* 等待按键松开 */
            while(READ_KEY_LEFT == 0);
            return KEY_LEFT_PRESS;
        }
        else if(READ_KEY_RIGHT == 0)
        {
            while(READ_KEY_RIGHT == 0);
            return KEY_RIGHT_PRESS;
        }
        else if(READ_KEY_UP == 0)
        {
            while(READ_KEY_UP == 0);
            return KEY_UP_PRESS;
        }
        else if(READ_KEY_DOWN == 0)
        {
            while(READ_KEY_DOWN == 0);
```

```
        return KEY_DOWN_PRESS;
    }

    else if(READ_KEY_LEFT_UP == 0)
    {
        uint16_t time = 0;
        while(READ_KEY_LEFT_UP == 0 && time < 1200)
        {
            common_delay_ms(1);
            time++;
        }
        while(READ_KEY_LEFT_UP == 0); /* 等待松开 */
        if(time < 500)                /* 松开之后判断是长按 or 短按 */
        {
            return KEY_LEFT_UP_PRESS;
        }
        else
        {
            return KEY_LEFT_UP_PRESS;
        }
    }

    else if(READ_KEY_RIGHT_UP == 0)
    {
        uint16_t time = 0;
        /* 等待按键松开 */
        while(READ_KEY_RIGHT_UP == 0 && time < 1200) /* 最多等待 1.2s */
        {
            common_delay_ms(1);
            time++;
        }

        while(READ_KEY_RIGHT_UP == 0); /* 等待松开 */

        if(time < 500) /* 在 500ms 松开，证明是一次普通的点按 */
        {
```

```
        return KEY_RIGHT_UP_PRESS;
    }

    else /* 超过 500ms 证明是长按 */
    {
        return KEY_RIGHT_UP_LONG_PRESS;
    }
}

return NOKEY_PRESS;
}
```

5.3 应用层

5.3.1 App_DataProcess.h

```
#ifndef __APP_DATAPROCESS_H
#define __APP_DATAPROCESS_H

#include "Common_Debug.h"
#include "Inf_JoyStickKey.h"

/* 对 x 限幅度处理 */
#define LIMIT(x, min, max) ((x) < (min) ? (min) : ((x) > (max) ? (max) : (x)))

/* 均值滤波缓冲区大小 */
#define FILTER_SIZE 10

/* 滤波缓冲区结构体 */
typedef struct
{
    int32_t values[10]; /* 存储最近的 10 个值 */
    int32_t index;      /* 索引 */
    int32_t sum;        /* 存储 10 个值的和 */
} FilterBuffer_Struct;

void App_DataProcess_Start(void);
```

```
void App_DataProcess_JoyStickDataProcess(void);

void App_DataProcess_KeyDataProcess(void);

#endif
```

5.3.2 App_DataProcess.c

```
#include "App_DataProcess.h"

/* 均值滤波(滑动窗口) */

#define MEAN_FILTER(filter, valuePointer) \
    filter.sum -= filter.values[filter.index]; \
    filter.values[filter.index] = *valuePointer; \
    filter.sum += filter.values[filter.index]; \
    filter.index = (filter.index + 1) % FILTER_SIZE; \
    *valuePointer = filter.sum / FILTER_SIZE

JoyStickBias_Struct JoyStickBias;

/**
 * @description: 摇杆和按键数据处理模块启动
 * @return {*}
 */

void App_DataProcess_Start(void)
{
    debug_printfln("摇杆和按键数据处理模块开始启动");

    Inf_JoyStickKey_Init();

    Inf_JoyStickKey_JoyStickScan();

    debug_printfln("摇杆和按键数据处理模块启动完成");
}

/**
 * @description: 摇杆极性数据处理
 * @return {*}
 */
```

```
static void App_DataProcess_JoyStickDataPolarity(void)
{
    /* 摇杆的极性，与我们的感知正好相反，先修改极性 */
    joyStick.THR = 1000 - joyStick.THR * 1000 / 4095;
    joyStick.YAW = 1000 - joyStick.YAW * 1000 / 4095;
    joyStick.PIT = 1000 - joyStick.PIT * 1000 / 4095;
    joyStick.ROL = 1000 - joyStick.ROL * 1000 / 4095;
}

/**
 * @description: 考虑摇杆的读取的值的偏移
 * @return {*}
 */
static void App_DataProcess_JoyStickDataWithBias(void)
{
    /* 为了防止出现负数，对移除偏移量的摇杆数据进行限幅处理 */
    joyStick.THR = LIMIT(joyStick.THR - JoyStickBias.THRBias, 0, 1000);
    joyStick.YAW = LIMIT(joyStick.YAW - JoyStickBias.YAWBias, 0, 1000);
    joyStick.PIT = LIMIT(joyStick.PIT - JoyStickBias.PITBias, 0, 1000);
    joyStick.ROL = LIMIT(joyStick.ROL - JoyStickBias.ROLBias, 0, 1000);
}

/* 初始化滤波缓冲区 */
static FilterBuffer_Struct THRFilter = {{0}, 0, 0};
static FilterBuffer_Struct YAWFilter = {{0}, 0, 0};
static FilterBuffer_Struct PITFilter = {{0}, 0, 0};
static FilterBuffer_Struct ROLFilter = {{0}, 0, 0};

/**
 * @description: 对摇杆 4 个方向数据进行滤波处理
 *
 *          这次我们使用平均值滤波(滑动窗口滤波)
 *
 *          取最近的 10 个值的平均值
 * @return {*}
 */
static void App_DataProcess_JoyStickDataFilter(void)
```

```
{

    /*

        // 1. 减去旧值

        THRFilter.sum -= THRFilter.values[THRFilter.index];

        // 2. 旧值的地方存储新值

        THRFilter.values[THRFilter.index] = joyStick.THR;

        // 3. 加上新值

        THRFilter.sum += THRFilter.values[THRFilter.index];

        // 4. 更新索引

        THRFilter.index = (THRFilter.index + 1) % FILTER_SIZE;

        //5. 计算均值

        joyStick.THR = THRFilter.sum / FILTER_SIZE;

        为了提高可读性,使用宏定义滤波算法

    */

    MEAN_FILTER(THRFilter, &joyStick.THR);
    MEAN_FILTER(YAWFilter, &joyStick.YAW);
    MEAN_FILTER(PITFilter, &joyStick.PIT);
    MEAN_FILTER(ROLFilter, &joyStick.ROL);
}

/**

 * @description: 处理摇杆数据

 * @return {*}

 */

void App_DataProcess_JoyStickDataProcess(void)

{

    Inf_JoyStickKey_JoyStickScan();           /* 扫描摇杆数据 */

    App_DataProcess_JoyStickDataPolarity(); /* 对摇杆数据进行极性处理 */

    App_DataProcess_JoyStickDataWithBias(); /* 考虑摇杆的偏差 */

    App_DataProcess_JoyStickDataFilter(); /* 对摇杆数据进行滤波 */

}
```

```
/**
 * @description: 对摇杆进行偏移量校准. 读取 100 次数据, 求出平均偏移量
 * 当长按右上按键的时候, 调用次函数进行校准
 * @return {*}
 */
static void App_DataProcess_JoyStickCalcBias()
{
    JoyStickBias.PITBias = 0;
    JoyStickBias.ROLBias = 0;
    JoyStickBias.YAWBias = 0;
    JoyStickBias.ROLBias = 0;
    /* 连续读取 100 次, 求取平均偏移量 */
    for(uint8_t i = 0; i < 100; i++)
    {
        Inf_JoyStickKey_JoyStickScan();          /* 扫描摇杆数据 */
        App_DataProcess_JoyStickDataPolarity(); /* 对摇杆数据进行极性处理 */
        JoyStickBias.THRBias += joyStick.THR - 0;
        JoyStickBias.YAWBias += joyStick.YAW - 500;
        JoyStickBias.PITBias += joyStick.PIT - 500;
        JoyStickBias.ROLBias += joyStick.ROL - 500;
    }

    JoyStickBias.THRBias /= 100;
    JoyStickBias.YAWBias /= 100;
    JoyStickBias.PITBias /= 100;
    JoyStickBias.ROLBias /= 100;
}

/**
 * @description: 按键数据处理:
 * @return {*}
 */
void App_DataProcess_KeyDataProcess(void)
```

```
{  
    /*  
        1. 如果是右上按键长按，则进行校准  
            THR 拉到最低，实现零值校准 0  
            其他由于会自动回中，则进行中值校准 500  
        4. 右上按键按一下启动定高定点  
        5. 左上按键按一下远程关闭飞控  
    */  
    uint8_t key = Inf_JoyStickKey_KeyScan();  
    if(key == KEY_RIGHT_UP_LONG_PRESS) /* 右上按键长安：实现零值校准 */  
    {  
        debug_printfln("长");  
        App_DataProcess_JoyStickCalcBias();  
    }  
    else if(key == KEY_LEFT_PRESS)  
    {  
        JoyStickBias.ROLBias += 10;  
    }  
    else if(key == KEY_UP_PRESS)  
    {  
        JoyStickBias.PITBias -= 10;  
    }  
    else if(key == KEY_RIGHT_PRESS)  
    {  
        JoyStickBias.ROLBias -= 10;  
    }  
    else if(key == KEY_DOWN_PRESS)  
    {  
        JoyStickBias.PITBias += 10;  
    }  
    else if(key == KEY_LEFT_UP_PRESS) /* 左上按键按下：远程关键 */  
    {  
        joyStick.isPowerDonw = 1;  
    }  
}
```

```
else if(key == KEY_RIGHT_UP_PRESS) /* 右上按键按下：关闭和开启定高定点模式 */
{
    debug_printfln("短");
    joyStick.isFixHeightPoint = 1;
}
}
```

5.4 任务调度

```
/* 3. 摇杆和按键 数据处理 */
void          dataProessTask(void *arg);
TaskHandle_t dataProessHandle;
#define JOYSTICK_PRIORITY 8
#define JOYSTICK_EXECUTE_CYCLE 4
/* 3. 摇杆和按键数据处理 */
void dataProessTask(void *arg)
{
    vTaskDelay(1000);
    debug_printfln("摇杆和按键处理任务：开始调度");
    TickType_t pxPreviousWakeTime = xTaskGetTickCount();
    while(1)
    {

        /* 1. 摇杆数据处理 */
        App_DataProcess_JoyStickDataProcess();

        /* 2. 按键处理 */
        App_DataProcess_KeyDataProcess();

        xTaskDelayUntil(&pxPreviousWakeTime, JOYSTICK_EXECUTE_CYCLE);
    }
}
```

第 6 章 通讯模块

6.1 STM32CubeMX 配置

参考飞控板配置

6.2 硬件接口层

参考飞控板

6.3 应用层

6.3.1 App_Communication.h

```
#ifndef __APP_COMMUNICATION_H
#define __APP_COMMUNICATION_H

#include "Common_Debug.h"
#include "Inf_Si24R1.h"
#include "Common_Delay.h"
#include "Common_Config.h"

void App_Communication_Start(void);
void App_Communication_SendJoyStickData(JoyStick_Struct *joyStick);
#endif
```

6.3.2 App_Communication.c

```
#include "App_Communication.h"

/**
 * @description: 启动通讯模块
 * @return {*}
 */
void App_Communication_Start(void)
{
    debug_printfln("通讯模块开始启动");
    /* 1. 等待自检通过 */
    debug_printfln("    2.4G 模块开始自检");
    HAL_Delay(1000);
    while(Inf_Si24R1_Check() == 1);
    debug_printfln("    2.4G 模块自检通过");
}
```

```
/* 2. 把 2.4G 模块设置为发送模式 */

debug_printfln("    2.4G 模块设置为发送模式");

Inf_Si24R1_TXMode();

debug_printfln("通讯模块完成启动");
}

/**
 * @description: 发送摇杆数据
 * 协议: 4 字节帧头+9 字节有效载荷(摇杆+定高)+4 字节校验和
 * @return {*}
 */
void App_Communication_SendJoyStickData(JoyStick_Struct *joyStick)
{
    /* 填充要发送的数据 */
    uint8_t index = 0;

    TX_BUFFER[index++] = FRAME_0; /* 0-3 为帧头 */
    TX_BUFFER[index++] = FRAME_1;
    TX_BUFFER[index++] = FRAME_2;
    TX_BUFFER[index++] = 0x00; /* 存储有效数据字节数 */
    TX_BUFFER[index++] = (joyStick->THR >> 8);
    TX_BUFFER[index++] = (joyStick->THR & 0xff);
    TX_BUFFER[index++] = (joyStick->YAW >> 8);
    TX_BUFFER[index++] = (joyStick->YAW & 0xff);
    TX_BUFFER[index++] = (joyStick->ROL >> 8);
    TX_BUFFER[index++] = (joyStick->ROL & 0xff);
    TX_BUFFER[index++] = (joyStick->PIT >> 8);
    TX_BUFFER[index++] = (joyStick->PIT & 0xff);

    TX_BUFFER[index++] = joyStick->isFixHeightPoint; /* 定高定点命令 0:不定高定点 1:定高定点 */
    TX_BUFFER[index++] = joyStick->isPowerDown; /* 关机命令 0:不关机 1:关机 */

    /* 关机命令只发一次即可。发完之后,把关机命令重新清零 */
}
```

```
joyStick->isPowerDown = 0;

joyStick->isFixHeightPoint = 0;

TX_BUFF[3] = index - 4; /* 有效载荷数 */

/* 存储 校验和，接收收到数据可用同样的方式进行校验*/
uint32_t checkSum = 0;
for(uint8_t i = 0; i < index; i++)
{
    checkSum += TX_BUFF[i];
}
TX_BUFF[index++] = (checkSum >> 24);
TX_BUFF[index++] = (checkSum >> 16);
TX_BUFF[index++] = (checkSum >> 8);
TX_BUFF[index++] = checkSum;

uint8_t r = Inf_Si24R1_TxPacket(TX_BUFF);
if(r == 0)
{
    // debug_printfln("发送成功 %d", checkSum);
}
else
{
    // debug_printfln("发送失败");
}
}
```

6.4 任务调度

```
/* 2. 通讯任务 */
void communicationTask(void *arg);
TaskHandle_t communicationHandle;
#define COMMUNICATION_PRIORITY 8
#define COMMUNICATION_EXECUTE_CYCLE 4
```

```
/* 2. 通讯任务执行函数 */
void communicationTask(void *arg)
{
    vTaskDelay(1000);
    debug_printfln("通讯任务：开始调度");
    TickType_t pxPreviousWakeTime = xTaskGetTickCount();
    while(1)
    {
        taskENTER_CRITICAL();
        App_Communication_SendJoyStickData(&joyStick);
        taskEXIT_CRITICAL();

        xTaskDelayUntil(&pxPreviousWakeTime, COMMUNICATION_EXECUTE_CYCLE);
    }
}
```

第 7 章 显示模块

7.1 硬件接口层

7.1.1 oled.h

```
#ifndef __OLED_H
#define __OLED_H

#include "main.h"
#include "gpio.h"

#define OLED_CS_GPIO GPIOA
#define OLED_CS_Pin GPIO_PIN_4

#define OLED_DC_GPIO GPIOB
#define OLED_DC_Pin GPIO_PIN_1

#define OLED_RST_GPIO GPIOB
#define OLED_RST_Pin GPIO_PIN_0
```

```
#define OLED_SDIN_GPIO GPIOA

#define OLED_SDIN_Pin GPIO_PIN_7

#define OLED_SCLK_GPIO GPIOA

#define OLED_SCLK_Pin GPIO_PIN_5

//-----OLED 端口定义-----

#define OLED_CS_Clr() HAL_GPIO_WritePin(GPIOA,GPIO_PIN_4,GPIO_PIN_RESET) // CS
#define OLED_CS_Set() HAL_GPIO_WritePin(GPIOA,GPIO_PIN_4,GPIO_PIN_SET) // CS

#define OLED_DC_Clr() HAL_GPIO_WritePin(GPIOB,GPIO_PIN_1,GPIO_PIN_RESET) // DC
#define OLED_DC_Set() HAL_GPIO_WritePin(GPIOB,GPIO_PIN_1,GPIO_PIN_SET) // DC

#define OLED_RST_Clr() HAL_GPIO_WritePin(GPIOB,GPIO_PIN_0,GPIO_PIN_RESET) // RST
#define OLED_RST_Set() HAL_GPIO_WritePin(GPIOB,GPIO_PIN_0,GPIO_PIN_SET) // RST

#define OLED_SDIN_Clr() HAL_GPIO_WritePin(GPIOA,GPIO_PIN_7,GPIO_PIN_RESET) // SDA
#define OLED_SDIN_Set() HAL_GPIO_WritePin(GPIOA,GPIO_PIN_7,GPIO_PIN_SET) // SDA

#define OLED_SCLK_Clr() HAL_GPIO_WritePin(GPIOA,GPIO_PIN_5,GPIO_PIN_RESET) // SCL
#define OLED_SCLK_Set() HAL_GPIO_WritePin(GPIOA,GPIO_PIN_5,GPIO_PIN_SET) // SCL

#define OLED_CMD 0 // 写命令
#define OLED_DATA 1 // 写数据

// OLED 控制用函数

void OLED_WR_Byte(uint8_t dat, uint8_t cmd);

void OLED_Display_On(void);

void OLED_Display_Off(void);

void OLED_Refresh_Gram(void);

void OLED_Init(void);

void OLED_Clear(void);

void OLED_DrawPoint(uint8_t x, uint8_t y, uint8_t t);
```

```
void OLED_ShowChar(uint8_t x, uint8_t y, uint8_t chr, uint8_t size, uint8_t mode);
void OLED_ShowNumber(uint8_t x, uint8_t y, uint32_t num, uint8_t len, uint8_t
size);
void OLED_ShowString(uint8_t x, uint8_t y, const uint8_t *p, uint8_t size, uint8_t
mode);
void OLED_ShowCH(uint8_t x, uint8_t y, uint8_t chr, uint8_t size, uint8_t mode);
void OLED_Show_CH(uint8_t x, uint8_t y, uint8_t chr, uint8_t size, uint8_t mode);
void OLED_Show_CH_String(uint8_t x, uint8_t y, const uint8_t *p, uint8_t size,
uint8_t mode);
#endif
```

7.1.2 oled.c

```
#include "oled.h"
#include "stdlib.h"
#include "oledfont.h"
#include "gpio.h"

uint8_t OLED_GRAM[128][8];
void OLED_Refresh_Gram(void)
{
    uint8_t i, n;
    for(i = 0; i < 8; i++)
    {
        OLED_WR_Byte(0xb0 + i, OLED_CMD); // 设置页地址 (0~7)
        OLED_WR_Byte(0x00, OLED_CMD); // 设置显示位置-列低地址
        OLED_WR_Byte(0x10, OLED_CMD); // 设置显示位置-列高地址
        for(n = 0; n < 128; n++)
            OLED_WR_Byte(OLED_GRAM[n][i], OLED_DATA);
    }
}

// 向 OLED 写入一个字节。
// dat:要写入的数据/命令
// cmd:数据/命令标志 0,表示命令;1,表示数据;
```

```
void OLED_WR_Byte(uint8_t dat, uint8_t cmd)
{
    uint8_t i;
    if(cmd)
        OLED_DC_Set();
    else
        OLED_DC_Clr();
    OLED_CS_Clr(); /* 要加: 先选中 */
    for(i = 0; i < 8; i++)
    {
        OLED_SCLK_Clr();
        if(dat & 0x80)
            OLED_SDIN_Set();
        else
            OLED_SDIN_Clr();
        OLED_SCLK_Set();
        dat <<= 1;
    }

    OLED_CS_Set(); /* 要加: 先释放 */
    OLED_DC_Set();
}

// 开启 OLED 显示
void OLED_Display_On(void)
{
    OLED_WR_Byte(0X8D, OLED_CMD); // SET DCDC 命令
    OLED_WR_Byte(0X14, OLED_CMD); // DCDC ON
    OLED_WR_Byte(0XAF, OLED_CMD); // DISPLAY ON
}

// 关闭 OLED 显示
void OLED_Display_Off(void)
{
    OLED_WR_Byte(0X8D, OLED_CMD); // SET DCDC 命令
```

```
OLED_WR_Byte(0X10, OLED_CMD);    // DCDC OFF

OLED_WR_Byte(0XAE, OLED_CMD);    // DISPLAY OFF
}

// 清屏函数,清完屏,整个屏幕是黑色的!和没点亮一样!!!
void OLED_Clear(void)
{
    uint8_t i, n;
    for(i = 0; i < 8; i++)
    {
        for(n = 0; n < 128; n++)
        {
            OLED_GRAM[n][i] = 0X00;
        }
    }

    OLED_Refresh_Gram();    // 更新显示
}

// 画点
// x:0~127
// y:0~63
// t:1 填充 0,清空

void OLED_DrawPoint(uint8_t x, uint8_t y, uint8_t t)
{
    uint8_t pos, bx, temp = 0;
    if(x > 127 || y > 63)
        return;    // 超出范围了.

    pos  = 7 - y / 8;
    bx   = y % 8;
    temp = 1 << (7 - bx);

    if(t)
        OLED_GRAM[x][pos] |= temp;
    else
        OLED_GRAM[x][pos] &= ~temp;
}
```

```
// 在指定位置显示一个字符,包括部分字符
// x:0~127
// y:0~63
// mode:0,反白显示;1,正常显示
// size:选择字体 16/12

void OLED_ShowChar(uint8_t x, uint8_t y, uint8_t chr, uint8_t size, uint8_t mode)
{
    uint8_t temp, t, t1;
    uint8_t y0 = y;
    chr      = chr - ' ';    // 得到偏移后的值
    for(t = 0; t < size; t++)
    {
        if(size == 12)
            temp = oled_asc2_1206[chr][t];    // 调用 1206 字体
        else
            temp = oled_asc2_1608[chr][t];    // 调用 1608 字体
        for(t1 = 0; t1 < 8; t1++)
        {
            if(temp & 0x80)
                OLED_DrawPoint(x, y, mode);
            else
                OLED_DrawPoint(x, y, !mode);
            temp <<= 1;
            y++;
            if((y - y0) == size)
            {
                y = y0;
                x++;
                break;
            }
        }
    }
}

// 显示汉字
```

```
void OLED_ShowCH(uint8_t x, uint8_t y, uint8_t chr, uint8_t size, uint8_t mode)
{
    uint8_t temp, t, t1;
    uint8_t y0 = y;

    for(t = 0; t < size; t++)
    {
        temp = oled_CH_1616[chr][t];    // 调用中文字库
        for(t1 = 0; t1 < 8; t1++)
        {
            if(temp & 0x80)
                OLED_DrawPoint(x, y, mode);
            else
                OLED_DrawPoint(x, y, !mode);
            temp <<= 1;
            y++;
            if((y - y0) == size)
            {
                y = y0;
                x++;
                break;
            }
        }
    }
}

// 显示单个中文字符
void OLED_Show_CH(uint8_t x, uint8_t y, uint8_t chr, uint8_t size, uint8_t mode)
{
    OLED_ShowCH(x, y, chr * 2, size, 1);
    OLED_ShowCH(x + size / 2, y, chr * 2 + 1, size, 1);
}

// 显示一串中文字符
```

```
void OLED_Show_CH_String(uint8_t x, uint8_t y, const uint8_t *p, uint8_t size,
uint8_t mode)
{
    uint8_t temp, t, t1;
    uint8_t y0 = y;

    for(t = 0; t < size; t++)
    {
        temp = p[t];    // 调用中文字库
        for(t1 = 0; t1 < 8; t1++)
        {
            if(temp & 0x80)
                OLED_DrawPoint(x, y, mode);
            else
                OLED_DrawPoint(x, y, !mode);
            temp <<= 1;
            y++;
            if((y - y0) == size)
            {
                y = y0;
                x++;
                break;
            }
        }
    }
}

// m^n 函数
uint32_t oled_pow(uint8_t m, uint8_t n)
{
    uint32_t result = 1;
    while(n--)
        result *= m;
    return result;
}
```

```
// 显示 2 个数字
// x,y :起点坐标
// len :数字的位数
// size:字体大小
// mode:模式 0,填充模式;1,叠加模式
// num:数值(0~4294967295);
void OLED_ShowNumber(uint8_t x, uint8_t y, uint32_t num, uint8_t len, uint8_t size)
{
    uint8_t t, temp;
    uint8_t enshow = 0;
    for(t = 0; t < len; t++)
    {
        temp = (num / oled_pow(10, len - t - 1)) % 10;
        if(enshow == 0 && t < (len - 1))
        {
            if(temp == 0)
            {
                OLED_ShowChar(x + (size / 2) * t, y, ' ', size, 1);
                continue;
            }
            else
            {
                enshow = 1;
            }
        }
        OLED_ShowChar(x + (size / 2) * t, y, temp + '0', size, 1);
    }
}

// 显示字符串
// x,y:起点坐标
// *p:字符串起始地址
// 用 16 字体
void OLED_ShowString(uint8_t x, uint8_t y, const uint8_t *p, uint8_t size, uint8_t
mode)
{
#define MAX_CHAR_POSX 122
```

```
#define MAX_CHAR_POSY 58

while(*p != '\0')
{
    if(x > MAX_CHAR_POSX)
    {
        x = 0;
        y += 16;
    } // 换行

    if(y > MAX_CHAR_POSY)
    {
        y = x = 0;
        // OLED_Clear();
    }

    OLED_ShowChar(x, y, *p, size, mode);
    x += 8;
    p++;
}

// 显示字符串
// x,y:起点坐标
// *p:字符串起始地址
// 用 16 字体

void OLED_ShowString_16(uint8_t x, uint8_t y, const uint8_t *p)
{
    #define MAX_CHAR_POSX 122
    #define MAX_CHAR_POSY 58

    while(*p != '\0')
    {
        if(x > MAX_CHAR_POSX)
        {
            x = 0;
            y += 16;
        } // 换行

        if(y > MAX_CHAR_POSY)
```

```
{
    y = x = 0;
    OLED_Clear();
}
OLED_ShowChar(x, y, *p, 16, 1);
x += 8;
p++;
}
}

// 初始化 OLED
void OLED_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure = {0};
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4 | GPIO_PIN_7, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0 | GPIO_PIN_1, GPIO_PIN_SET);
    GPIO_InitStructure.Pin = GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_7;
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.Pin = GPIO_PIN_0 | GPIO_PIN_1;
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStructure);

    OLED_RST_Clr();
    HAL_Delay(100);
    OLED_RST_Set();

    OLED_WR_Byte(0xAE, OLED_CMD); // --turn off oled panel
```

```
OLED_WR_Byte(0x00, OLED_CMD); //---set low column address
OLED_WR_Byte(0x10, OLED_CMD); //---set high column address
OLED_WR_Byte(0x40, OLED_CMD); //---set start line address Set Mapping RAM
Display Start Line (0x00~0x3F)
OLED_WR_Byte(0x81, OLED_CMD); //---set contrast control register
OLED_WR_Byte(0xCF, OLED_CMD); // Set SEG Output Current Brightness
OLED_WR_Byte(0xA1, OLED_CMD); //---Set SEG/Column Mapping 0xa0 左右反置 0xa1
正常
OLED_WR_Byte(0xC0, OLED_CMD); // Set COM/Row Scan Direction 0xc0 上下反置
0xc8 正常
OLED_WR_Byte(0xA6, OLED_CMD); //---set normal display
OLED_WR_Byte(0xA8, OLED_CMD); //---set multiplex ratio(1 to 64)
OLED_WR_Byte(0x3f, OLED_CMD); //---1/64 duty
OLED_WR_Byte(0xD3, OLED_CMD); //---set display offset Shift Mapping RAM
Counter (0x00~0x3F)
OLED_WR_Byte(0x00, OLED_CMD); //---not offset
OLED_WR_Byte(0xd5, OLED_CMD); //---set display clock divide ratio/oscillator
frequency
OLED_WR_Byte(0x80, OLED_CMD); //---set divide ratio, Set Clock as 100
Frames/Sec
OLED_WR_Byte(0xD9, OLED_CMD); //---set pre-charge period
OLED_WR_Byte(0xF1, OLED_CMD); // Set Pre-Charge as 15 Clocks & Discharge as 1
Clock
OLED_WR_Byte(0xDA, OLED_CMD); //---set com pins hardware configuration
OLED_WR_Byte(0x12, OLED_CMD);
OLED_WR_Byte(0xDB, OLED_CMD); //---set vcomh
OLED_WR_Byte(0x30, OLED_CMD); // Set VCOM Deselect Level
OLED_WR_Byte(0x20, OLED_CMD); //---Set Page Addressing Mode (0x00/0x01/0x02)
OLED_WR_Byte(0x02, OLED_CMD); //
OLED_WR_Byte(0x8D, OLED_CMD); //---set Charge Pump enable/disable
OLED_WR_Byte(0x14, OLED_CMD); //---set(0x10) disable
OLED_Clear();
OLED_WR_Byte(0xAF, OLED_CMD);
}
```

7.1.3 oledfont.h

```
#ifndef __OLEDFONT_H
#define __OLEDFONT_H

extern const unsigned char oled_asc2_1206[95][12];
extern const unsigned char oled_asc2_1608[95][16];
extern const unsigned char oled_CH_1616[][12];

extern const unsigned char oled_CH0[][12];
extern const unsigned char oled_CH1[][12];
extern const unsigned char oled_CH2[][12];
extern const unsigned char oled_CH3[][12];
extern const unsigned char oled_CH4[][12];
extern const unsigned char oled_CH5[][12];
extern const unsigned char oled_CH6[][12];
extern const unsigned char oled_CH7[][12];
extern const unsigned char oled_CH8[][12];
extern const unsigned char oled_CH9[][12];

extern const unsigned char oled_CH10[][12];
extern const unsigned char oled_CH11[][12];
extern const unsigned char oled_CH12[][12];
extern const unsigned char oled_CH13[][12];
extern const unsigned char oled_CH14[][12];
extern const unsigned char oled_CH15[][12];
extern const unsigned char oled_CH16[][12];
extern const unsigned char oled_CH17[][12];
extern const unsigned char oled_CH18[][12];
extern const unsigned char oled_CH19[][12];

extern const unsigned char oled_CH20[][12];

#endif
```

7.1.4 oledfont.c

```
#include "oledfont.h"

//常用 ASCII 表
//偏移量 32
//ASCII 字符集
//偏移量 32
//大小:12*6

const unsigned char oled_asc2_1206[95][12]={
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/*" ",0*/
{0x00,0x00,0x00,0x00,0x3F,0x40,0x00,0x00,0x00,0x00,0x00,0x00},/*"! ",1*/
{0x00,0x00,0x30,0x00,0x40,0x00,0x30,0x00,0x40,0x00,0x00,0x00},/*"\"",2*/
{0x09,0x00,0x0B,0xC0,0x3D,0x00,0x0B,0xC0,0x3D,0x00,0x09,0x00},/*"#",3*/
{0x18,0xC0,0x24,0x40,0x7F,0xE0,0x22,0x40,0x31,0x80,0x00,0x00},/*"$",4*/
{0x18,0x00,0x24,0xC0,0x1B,0x00,0x0D,0x80,0x32,0x40,0x01,0x80},/*"%",5*/
{0x03,0x80,0x1C,0x40,0x27,0x40,0x1C,0x80,0x07,0x40,0x00,0x40},/*"&",6*/
{0x10,0x00,0x60,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/*"' ",7*/
{0x00,0x00,0x00,0x00,0x00,0x00,0x1F,0x80,0x20,0x40,0x40,0x20},/*"(",8*/
{0x00,0x00,0x40,0x20,0x20,0x40,0x1F,0x80,0x00,0x00,0x00,0x00},/*")",9*/
{0x09,0x00,0x06,0x00,0x1F,0x80,0x06,0x00,0x09,0x00,0x00,0x00},/*"*",10*/
{0x04,0x00,0x04,0x00,0x3F,0x80,0x04,0x00,0x04,0x00,0x00,0x00},/*"+",11*/
{0x00,0x10,0x00,0x60,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/*",",12*/
{0x04,0x00,0x04,0x00,0x04,0x00,0x04,0x00,0x04,0x00,0x00,0x00},/*"- ",13*/
{0x00,0x00,0x00,0x40,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/*".",14*/
{0x00,0x20,0x01,0xC0,0x06,0x00,0x38,0x00,0x40,0x00,0x00,0x00},/*"/",15*/
{0x1F,0x80,0x20,0x40,0x20,0x40,0x20,0x40,0x1F,0x80,0x00,0x00},/*"0",16*/
{0x00,0x00,0x10,0x40,0x3F,0xC0,0x00,0x40,0x00,0x00,0x00,0x00},/*"1",17*/
{0x18,0xC0,0x21,0x40,0x22,0x40,0x24,0x40,0x18,0x40,0x00,0x00},/*"2",18*/
{0x10,0x80,0x20,0x40,0x24,0x40,0x24,0x40,0x1B,0x80,0x00,0x00},/*"3",19*/
{0x02,0x00,0x0D,0x00,0x11,0x00,0x3F,0xC0,0x01,0x40,0x00,0x00},/*"4",20*/
{0x3C,0x80,0x24,0x40,0x24,0x40,0x24,0x40,0x23,0x80,0x00,0x00},/*"5",21*/
{0x1F,0x80,0x24,0x40,0x24,0x40,0x34,0x40,0x03,0x80,0x00,0x00},/*"6",22*/
{0x30,0x00,0x20,0x00,0x27,0xC0,0x38,0x00,0x20,0x00,0x00,0x00},/*"7",23*/
```

```
{0x1B,0x80,0x24,0x40,0x24,0x40,0x24,0x40,0x1B,0x80,0x00,0x00},/*"8",24*/
{0x1C,0x00,0x22,0xC0,0x22,0x40,0x22,0x40,0x1F,0x80,0x00,0x00},/*"9",25*/
{0x00,0x00,0x00,0x00,0x08,0x40,0x00,0x00,0x00,0x00,0x00},/*":",26*/
{0x00,0x00,0x00,0x00,0x04,0x60,0x00,0x00,0x00,0x00,0x00},/*";",27*/
{0x00,0x00,0x04,0x00,0x0A,0x00,0x11,0x00,0x20,0x80,0x40,0x40},/*"<",28*/
{0x09,0x00,0x09,0x00,0x09,0x00,0x09,0x00,0x09,0x00,0x00},/*"=",29*/
{0x00,0x00,0x40,0x40,0x20,0x80,0x11,0x00,0x0A,0x00,0x04,0x00},/*">",30*/
{0x18,0x00,0x20,0x00,0x23,0x40,0x24,0x00,0x18,0x00,0x00,0x00},/*"?",31*/
{0x1F,0x80,0x20,0x40,0x27,0x40,0x29,0x40,0x1F,0x40,0x00,0x00},/*"@",32*/
{0x00,0x40,0x07,0xC0,0x39,0x00,0x0F,0x00,0x01,0xC0,0x00,0x40},/*"A",33*/
{0x20,0x40,0x3F,0xC0,0x24,0x40,0x24,0x40,0x1B,0x80,0x00,0x00},/*"B",34*/
{0x1F,0x80,0x20,0x40,0x20,0x40,0x20,0x40,0x30,0x80,0x00,0x00},/*"C",35*/
{0x20,0x40,0x3F,0xC0,0x20,0x40,0x20,0x40,0x1F,0x80,0x00,0x00},/*"D",36*/
{0x20,0x40,0x3F,0xC0,0x24,0x40,0x2E,0x40,0x30,0xC0,0x00,0x00},/*"E",37*/
{0x20,0x40,0x3F,0xC0,0x24,0x40,0x2E,0x00,0x30,0x00,0x00,0x00},/*"F",38*/
{0x0F,0x00,0x10,0x80,0x20,0x40,0x22,0x40,0x33,0x80,0x02,0x00},/*"G",39*/
{0x20,0x40,0x3F,0xC0,0x04,0x00,0x04,0x00,0x3F,0xC0,0x20,0x40},/*"H",40*/
{0x20,0x40,0x20,0x40,0x3F,0xC0,0x20,0x40,0x20,0x40,0x00,0x00},/*"I",41*/
{0x00,0x60,0x20,0x20,0x20,0x20,0x3F,0xC0,0x20,0x00,0x20,0x00},/*"J",42*/
{0x20,0x40,0x3F,0xC0,0x24,0x40,0x0B,0x00,0x30,0xC0,0x20,0x40},/*"K",43*/
{0x20,0x40,0x3F,0xC0,0x20,0x40,0x00,0x40,0x00,0x40,0x00,0xC0},/*"L",44*/
{0x3F,0xC0,0x3C,0x00,0x03,0xC0,0x3C,0x00,0x3F,0xC0,0x00,0x00},/*"M",45*/
{0x20,0x40,0x3F,0xC0,0x0C,0x40,0x23,0x00,0x3F,0xC0,0x20,0x00},/*"N",46*/
{0x1F,0x80,0x20,0x40,0x20,0x40,0x20,0x40,0x1F,0x80,0x00,0x00},/*"O",47*/
{0x20,0x40,0x3F,0xC0,0x24,0x40,0x24,0x00,0x18,0x00,0x00,0x00},/*"P",48*/
{0x1F,0x80,0x21,0x40,0x21,0x40,0x20,0xE0,0x1F,0xA0,0x00,0x00},/*"Q",49*/
{0x20,0x40,0x3F,0xC0,0x24,0x40,0x26,0x00,0x19,0xC0,0x00,0x40},/*"R",50*/
{0x18,0xC0,0x24,0x40,0x24,0x40,0x22,0x40,0x31,0x80,0x00,0x00},/*"S",51*/
{0x30,0x00,0x20,0x40,0x3F,0xC0,0x20,0x40,0x30,0x00,0x00,0x00},/*"T",52*/
{0x20,0x00,0x3F,0x80,0x00,0x40,0x00,0x40,0x3F,0x80,0x20,0x00},/*"U",53*/
{0x20,0x00,0x3E,0x00,0x01,0xC0,0x07,0x00,0x38,0x00,0x20,0x00},/*"V",54*/
{0x38,0x00,0x07,0xC0,0x3C,0x00,0x07,0xC0,0x38,0x00,0x00,0x00},/*"W",55*/
{0x20,0x40,0x39,0xC0,0x06,0x00,0x39,0xC0,0x20,0x40,0x00,0x00},/*"X",56*/
{0x20,0x00,0x38,0x40,0x07,0xC0,0x38,0x40,0x20,0x00,0x00,0x00},/*"Y",57*/
```

```
{0x30,0x40,0x21,0xC0,0x26,0x40,0x38,0x40,0x20,0xC0,0x00,0x00},/*"Z",58*/
{0x00,0x00,0x00,0x00,0x7F,0xE0,0x40,0x20,0x40,0x20,0x00,0x00},/*"[",59*/
{0x00,0x00,0x70,0x00,0x0C,0x00,0x03,0x80,0x00,0x40,0x00,0x00},/*"\",60*/
{0x00,0x00,0x40,0x20,0x40,0x20,0x7F,0xE0,0x00,0x00,0x00,0x00},/*"]",61*/
{0x00,0x00,0x20,0x00,0x40,0x00,0x20,0x00,0x00,0x00,0x00,0x00},/*"^",62*/
{0x00,0x10,0x00,0x10,0x00,0x10,0x00,0x10,0x00,0x10,0x00,0x10},/*"_",63*/
{0x00,0x00,0x00,0x00,0x40,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/*"`",64*/
{0x00,0x00,0x02,0x80,0x05,0x40,0x05,0x40,0x03,0xC0,0x00,0x40},/*"a",65*/
{0x20,0x00,0x3F,0xC0,0x04,0x40,0x04,0x40,0x03,0x80,0x00,0x00},/*"b",66*/
{0x00,0x00,0x03,0x80,0x04,0x40,0x04,0x40,0x06,0x40,0x00,0x00},/*"c",67*/
{0x00,0x00,0x03,0x80,0x04,0x40,0x24,0x40,0x3F,0xC0,0x00,0x40},/*"d",68*/
{0x00,0x00,0x03,0x80,0x05,0x40,0x05,0x40,0x03,0x40,0x00,0x00},/*"e",69*/
{0x00,0x00,0x04,0x40,0x1F,0xC0,0x24,0x40,0x24,0x40,0x20,0x00},/*"f",70*/
{0x00,0x00,0x02,0xE0,0x05,0x50,0x05,0x50,0x06,0x50,0x04,0x20},/*"g",71*/
{0x20,0x40,0x3F,0xC0,0x04,0x40,0x04,0x00,0x03,0xC0,0x00,0x40},/*"h",72*/
{0x00,0x00,0x04,0x40,0x27,0xC0,0x00,0x40,0x00,0x00,0x00,0x00},/*"i",73*/
{0x00,0x10,0x00,0x10,0x04,0x10,0x27,0xE0,0x00,0x00,0x00,0x00},/*"j",74*/
{0x20,0x40,0x3F,0xC0,0x01,0x40,0x07,0x00,0x04,0xC0,0x04,0x40},/*"k",75*/
{0x20,0x40,0x20,0x40,0x3F,0xC0,0x00,0x40,0x00,0x40,0x00,0x00},/*"l",76*/
{0x07,0xC0,0x04,0x00,0x07,0xC0,0x04,0x00,0x03,0xC0,0x00,0x00},/*"m",77*/
{0x04,0x40,0x07,0xC0,0x04,0x40,0x04,0x00,0x03,0xC0,0x00,0x40},/*"n",78*/
{0x00,0x00,0x03,0x80,0x04,0x40,0x04,0x40,0x03,0x80,0x00,0x00},/*"o",79*/
{0x04,0x10,0x07,0xF0,0x04,0x50,0x04,0x40,0x03,0x80,0x00,0x00},/*"p",80*/
{0x00,0x00,0x03,0x80,0x04,0x40,0x04,0x50,0x07,0xF0,0x00,0x10},/*"q",81*/
{0x04,0x40,0x07,0xC0,0x02,0x40,0x04,0x00,0x04,0x00,0x00,0x00},/*"r",82*/
{0x00,0x00,0x06,0x40,0x05,0x40,0x05,0x40,0x04,0xC0,0x00,0x00},/*"s",83*/
{0x00,0x00,0x04,0x00,0x1F,0x80,0x04,0x40,0x00,0x40,0x00,0x00},/*"t",84*/
{0x04,0x00,0x07,0x80,0x00,0x40,0x04,0x40,0x07,0xC0,0x00,0x40},/*"u",85*/
{0x04,0x00,0x07,0x00,0x04,0xC0,0x01,0x80,0x06,0x00,0x04,0x00},/*"v",86*/
{0x06,0x00,0x01,0xC0,0x07,0x00,0x01,0xC0,0x06,0x00,0x00,0x00},/*"w",87*/
{0x04,0x40,0x06,0xC0,0x01,0x00,0x06,0xC0,0x04,0x40,0x00,0x00},/*"x",88*/
{0x04,0x10,0x07,0x10,0x04,0xE0,0x01,0x80,0x06,0x00,0x04,0x00},/*"y",89*/
{0x00,0x00,0x04,0x40,0x05,0xC0,0x06,0x40,0x04,0x40,0x00,0x00},/*"z",90*/
{0x00,0x00,0x00,0x00,0x04,0x00,0x7B,0xE0,0x40,0x20,0x00,0x00},/*"{",91*/
```

```
{0x00,0x00,0x00,0x00,0x00,0x00,0xFF,0xF0,0x00,0x00,0x00,0x00},/*"|",92*/
{0x00,0x00,0x40,0x20,0x7B,0xE0,0x04,0x00,0x00,0x00,0x00,0x00},/*"}",93*/
{0x40,0x00,0x80,0x00,0x40,0x00,0x20,0x00,0x20,0x00,0x40,0x00},/*"~",94*/
};

const unsigned char oled_asc2_1608[95][16]={
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/
*" ",0*/
{0x00,0x00,0x00,0x00,0x00,0x00,0x1F,0xCC,0x00,0x0C,0x00,0x00,0x00,0x00,0x00,0x00},/
*"!",1*/
{0x00,0x00,0x08,0x00,0x30,0x00,0x60,0x00,0x08,0x00,0x30,0x00,0x60,0x00,0x00,0x00},/
*"\"",2*/
{0x02,0x20,0x03,0xFC,0x1E,0x20,0x02,0x20,0x03,0xFC,0x1E,0x20,0x02,0x20,0x00,0x00},/
*"#",3*/
{0x00,0x00,0x0E,0x18,0x11,0x04,0x3F,0xFF,0x10,0x84,0x0C,0x78,0x00,0x00,0x00,0x00},/
*"$",4*/
{0x0F,0x00,0x10,0x84,0x0F,0x38,0x00,0xC0,0x07,0x78,0x18,0x84,0x00,0x78,0x00,0x00},/
*"%",5*/
{0x00,0x78,0x0F,0x84,0x10,0xC4,0x11,0x24,0x0E,0x98,0x00,0xE4,0x00,0x84,0x00,0x08},/
*"&",6*/
{0x08,0x00,0x68,0x00,0x70,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/
*"'",7*/
{0x00,0x00,0x00,0x00,0x00,0x00,0x07,0xE0,0x18,0x18,0x20,0x04,0x40,0x02,0x00,0x00},/
*"(",8*/
{0x00,0x00,0x40,0x02,0x20,0x04,0x18,0x18,0x07,0xE0,0x00,0x00,0x00,0x00,0x00,0x00},/
*)"",9*/
{0x02,0x40,0x02,0x40,0x01,0x80,0x0F,0xF0,0x01,0x80,0x02,0x40,0x02,0x40,0x00,0x00},/
*"*",10*/
{0x00,0x80,0x00,0x80,0x00,0x80,0x0F,0xF8,0x00,0x80,0x00,0x80,0x00,0x80,0x00,0x00},/
*"+",11*/
{0x00,0x01,0x00,0x0D,0x00,0x0E,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/
*",",12*/
{0x00,0x00,0x00,0x80,0x00,0x80,0x00,0x80,0x00,0x80,0x00,0x80,0x00,0x80,0x00,0x80},/
*"-",13*/
```

```
{0x00,0x00,0x00,0x0C,0x00,0x0C,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/  
*".",14*/  
  
{0x00,0x00,0x00,0x06,0x00,0x18,0x00,0x60,0x01,0x80,0x06,0x00,0x18,0x00,0x20,0x00},/  
*"/",15*/  
  
{0x00,0x00,0x07,0xF0,0x08,0x08,0x10,0x04,0x10,0x04,0x08,0x08,0x07,0xF0,0x00,0x00},/  
*"0",16*/  
  
{0x00,0x00,0x08,0x04,0x08,0x04,0x1F,0xFC,0x00,0x04,0x00,0x04,0x00,0x00,0x00,0x00},/  
*"1",17*/  
  
{0x00,0x00,0x0E,0x0C,0x10,0x14,0x10,0x24,0x10,0x44,0x11,0x84,0x0E,0x0C,0x00,0x00},/  
*"2",18*/  
  
{0x00,0x00,0x0C,0x18,0x10,0x04,0x11,0x04,0x11,0x04,0x12,0x88,0x0C,0x70,0x00,0x00},/  
*"3",19*/  
  
{0x00,0x00,0x00,0xE0,0x03,0x20,0x04,0x24,0x08,0x24,0x1F,0xFC,0x00,0x24,0x00,0x00},/  
*"4",20*/  
  
{0x00,0x00,0x1F,0x98,0x10,0x84,0x11,0x04,0x11,0x04,0x10,0x88,0x10,0x70,0x00,0x00},/  
*"5",21*/  
  
{0x00,0x00,0x07,0xF0,0x08,0x88,0x11,0x04,0x11,0x04,0x18,0x88,0x00,0x70,0x00,0x00},/  
*"6",22*/  
  
{0x00,0x00,0x1C,0x00,0x10,0x00,0x10,0xFC,0x13,0x00,0x1C,0x00,0x10,0x00,0x00,0x00},/  
*"7",23*/  
  
{0x00,0x00,0x0E,0x38,0x11,0x44,0x10,0x84,0x10,0x84,0x11,0x44,0x0E,0x38,0x00,0x00},/  
*"8",24*/  
  
{0x00,0x00,0x07,0x00,0x08,0x8C,0x10,0x44,0x10,0x44,0x08,0x88,0x07,0xF0,0x00,0x00},/  
*"9",25*/  
  
{0x00,0x00,0x00,0x00,0x00,0x00,0x03,0x0C,0x03,0x0C,0x00,0x00,0x00,0x00,0x00,0x00},/  
*":",26*/  
  
{0x00,0x00,0x00,0x00,0x00,0x01,0x01,0x06,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/  
*";",27*/  
  
{0x00,0x00,0x00,0x80,0x01,0x40,0x02,0x20,0x04,0x10,0x08,0x08,0x10,0x04,0x00,0x00},/  
*"("<,28*/  
  
{0x02,0x20,0x02,0x20,0x02,0x20,0x02,0x20,0x02,0x20,0x02,0x20,0x02,0x20,0x00,0x00},/  
*"=",29*/  
  
{0x00,0x00,0x10,0x04,0x08,0x08,0x04,0x10,0x02,0x20,0x01,0x40,0x00,0x80,0x00,0x00},/  
*"(">,30*/
```

```
{0x00,0x00,0x0E,0x00,0x12,0x00,0x10,0x0C,0x10,0x6C,0x10,0x80,0x0F,0x00,0x00,0x00},/  
*"?",31*/  
  
{0x03,0xE0,0x0C,0x18,0x13,0xE4,0x14,0x24,0x17,0xC4,0x08,0x28,0x07,0xD0,0x00,0x00},/  
*"@",32*/  
  
{0x00,0x04,0x00,0x3C,0x03,0xC4,0x1C,0x40,0x07,0x40,0x00,0xE4,0x00,0x1C,0x00,0x04},/  
*"A",33*/  
  
{0x10,0x04,0x1F,0xFC,0x11,0x04,0x11,0x04,0x11,0x04,0x0E,0x88,0x00,0x70,0x00,0x00},/  
*"B",34*/  
  
{0x03,0xE0,0x0C,0x18,0x10,0x04,0x10,0x04,0x10,0x04,0x10,0x08,0x1C,0x10,0x00,0x00},/  
*"C",35*/  
  
{0x10,0x04,0x1F,0xFC,0x10,0x04,0x10,0x04,0x10,0x04,0x08,0x08,0x07,0xF0,0x00,0x00},/  
*"D",36*/  
  
{0x10,0x04,0x1F,0xFC,0x11,0x04,0x11,0x04,0x17,0xC4,0x10,0x04,0x08,0x18,0x00,0x00},/  
*"E",37*/  
  
{0x10,0x04,0x1F,0xFC,0x11,0x04,0x11,0x00,0x17,0xC0,0x10,0x00,0x08,0x00,0x00,0x00},/  
*"F",38*/  
  
{0x03,0xE0,0x0C,0x18,0x10,0x04,0x10,0x04,0x10,0x44,0x1C,0x78,0x00,0x40,0x00,0x00},/  
*"G",39*/  
  
{0x10,0x04,0x1F,0xFC,0x10,0x84,0x00,0x80,0x00,0x80,0x10,0x84,0x1F,0xFC,0x10,0x04},/  
*"H",40*/  
  
{0x00,0x00,0x10,0x04,0x10,0x04,0x1F,0xFC,0x10,0x04,0x10,0x04,0x00,0x00,0x00,0x00},/  
*"I",41*/  
  
{0x00,0x03,0x00,0x01,0x10,0x01,0x10,0x01,0x1F,0xFE,0x10,0x00,0x10,0x00,0x00,0x00},/  
*"J",42*/  
  
{0x10,0x04,0x1F,0xFC,0x11,0x04,0x03,0x80,0x14,0x64,0x18,0x1C,0x10,0x04,0x00,0x00},/  
*"K",43*/  
  
{0x10,0x04,0x1F,0xFC,0x10,0x04,0x00,0x04,0x00,0x04,0x00,0x04,0x00,0x0C,0x00,0x00},/  
*"L",44*/  
  
{0x10,0x04,0x1F,0xFC,0x1F,0x00,0x00,0xFC,0x1F,0x00,0x1F,0xFC,0x10,0x04,0x00,0x00},/  
*"M",45*/  
  
{0x10,0x04,0x1F,0xFC,0x0C,0x04,0x03,0x00,0x00,0xE0,0x10,0x18,0x1F,0xFC,0x10,0x00},/  
*"N",46*/  
  
{0x07,0xF0,0x08,0x08,0x10,0x04,0x10,0x04,0x10,0x04,0x08,0x08,0x07,0xF0,0x00,0x00},/  
*"O",47*/
```

```
{0x10,0x04,0x1F,0xFC,0x10,0x84,0x10,0x80,0x10,0x80,0x10,0x80,0x0F,0x00,0x00,0x00},/  
*"P",48*/  
  
{0x07,0xF0,0x08,0x18,0x10,0x24,0x10,0x24,0x10,0x1C,0x08,0x0A,0x07,0xF2,0x00,0x00},/  
*"Q",49*/  
  
{0x10,0x04,0x1F,0xFC,0x11,0x04,0x11,0x00,0x11,0xC0,0x11,0x30,0x0E,0x0C,0x00,0x04},/  
*"R",50*/  
  
{0x00,0x00,0x0E,0x1C,0x11,0x04,0x10,0x84,0x10,0x84,0x10,0x44,0x1C,0x38,0x00,0x00},/  
*"S",51*/  
  
{0x18,0x00,0x10,0x00,0x10,0x04,0x1F,0xFC,0x10,0x04,0x10,0x00,0x18,0x00,0x00,0x00},/  
*"T",52*/  
  
{0x10,0x00,0x1F,0xF8,0x10,0x04,0x00,0x04,0x00,0x04,0x10,0x04,0x1F,0xF8,0x10,0x00},/  
*"U",53*/  
  
{0x10,0x00,0x1E,0x00,0x11,0xE0,0x00,0x1C,0x00,0x70,0x13,0x80,0x1C,0x00,0x10,0x00},/  
*"V",54*/  
  
{0x1F,0xC0,0x10,0x3C,0x00,0xE0,0x1F,0x00,0x00,0xE0,0x10,0x3C,0x1F,0xC0,0x00,0x00},/  
*"W",55*/  
  
{0x10,0x04,0x18,0x0C,0x16,0x34,0x01,0xC0,0x01,0xC0,0x16,0x34,0x18,0x0C,0x10,0x04},/  
*"X",56*/  
  
{0x10,0x00,0x1C,0x00,0x13,0x04,0x00,0xFC,0x13,0x04,0x1C,0x00,0x10,0x00,0x00,0x00},/  
*"Y",57*/  
  
{0x08,0x04,0x10,0x1C,0x10,0x64,0x10,0x84,0x13,0x04,0x1C,0x04,0x10,0x18,0x00,0x00},/  
*"Z",58*/  
  
{0x00,0x00,0x00,0x00,0x00,0x00,0x7F,0xFE,0x40,0x02,0x40,0x02,0x40,0x02,0x00,0x00},/  
*"[",59*/  
  
{0x00,0x00,0x30,0x00,0x0C,0x00,0x03,0x80,0x00,0x60,0x00,0x1C,0x00,0x03,0x00,0x00},/  
*"\",60*/  
  
{0x00,0x00,0x40,0x02,0x40,0x02,0x40,0x02,0x7F,0xFE,0x00,0x00,0x00,0x00,0x00,0x00},/  
*"]",61*/  
  
{0x00,0x00,0x00,0x00,0x20,0x00,0x40,0x00,0x40,0x00,0x40,0x00,0x20,0x00,0x00,0x00},/  
*"^",62*/  
  
{0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x01},/  
*"_",63*/  
  
{0x00,0x00,0x40,0x00,0x40,0x00,0x20,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/  
*"`,64*/
```

```
{0x00,0x00,0x00,0x98,0x01,0x24,0x01,0x44,0x01,0x44,0x01,0x44,0x00,0xFC,0x00,0x04},/  
*"a",65*/  
  
{0x10,0x00,0x1F,0xFC,0x00,0x88,0x01,0x04,0x01,0x04,0x00,0x88,0x00,0x70,0x00,0x00},/  
*"b",66*/  
  
{0x00,0x00,0x00,0x70,0x00,0x88,0x01,0x04,0x01,0x04,0x01,0x04,0x00,0x88,0x00,0x00},/  
*"c",67*/  
  
{0x00,0x00,0x00,0x70,0x00,0x88,0x01,0x04,0x01,0x04,0x11,0x08,0x1F,0xFC,0x00,0x04},/  
*"d",68*/  
  
{0x00,0x00,0x00,0xF8,0x01,0x44,0x01,0x44,0x01,0x44,0x01,0x44,0x00,0xC8,0x00,0x00},/  
*"e",69*/  
  
{0x00,0x00,0x01,0x04,0x01,0x04,0x0F,0xFC,0x11,0x04,0x11,0x04,0x11,0x00,0x18,0x00},/  
*"f",70*/  
  
{0x00,0x00,0x00,0xD6,0x01,0x29,0x01,0x29,0x01,0x29,0x01,0xC9,0x01,0x06,0x00,0x00},/  
*"g",71*/  
  
{0x10,0x04,0x1F,0xFC,0x00,0x84,0x01,0x00,0x01,0x00,0x01,0x04,0x00,0xFC,0x00,0x04},/  
*"h",72*/  
  
{0x00,0x00,0x01,0x04,0x19,0x04,0x19,0xFC,0x00,0x04,0x00,0x04,0x00,0x00,0x00,0x00},/  
*"i",73*/  
  
{0x00,0x00,0x00,0x03,0x00,0x01,0x01,0x01,0x19,0x01,0x19,0xFE,0x00,0x00,0x00,0x00},/  
*"j",74*/  
  
{0x10,0x04,0x1F,0xFC,0x00,0x24,0x00,0x40,0x01,0xB4,0x01,0x0C,0x01,0x04,0x00,0x00},/  
*"k",75*/  
  
{0x00,0x00,0x10,0x04,0x10,0x04,0x1F,0xFC,0x00,0x04,0x00,0x04,0x00,0x00,0x00,0x00},/  
*"l",76*/  
  
{0x01,0x04,0x01,0xFC,0x01,0x04,0x01,0x00,0x01,0xFC,0x01,0x04,0x01,0x00,0x00,0xFC},/  
*"m",77*/  
  
{0x01,0x04,0x01,0xFC,0x00,0x84,0x01,0x00,0x01,0x00,0x01,0x04,0x00,0xFC,0x00,0x04},/  
*"n",78*/  
  
{0x00,0x00,0x00,0xF8,0x01,0x04,0x01,0x04,0x01,0x04,0x01,0x04,0x00,0xF8,0x00,0x00},/  
*"o",79*/  
  
{0x01,0x01,0x01,0xFF,0x00,0x85,0x01,0x04,0x01,0x04,0x00,0x88,0x00,0x70,0x00,0x00},/  
*"p",80*/  
  
{0x00,0x00,0x00,0x70,0x00,0x88,0x01,0x04,0x01,0x04,0x01,0x05,0x01,0xFF,0x00,0x01},/  
*"q",81*/
```

```
{0x01,0x04,0x01,0x04,0x01,0xFC,0x00,0x84,0x01,0x04,0x01,0x00,0x01,0x80,0x00,0x00},/  
*"r",82*/  
  
{0x00,0x00,0x00,0xCC,0x01,0x24,0x01,0x24,0x01,0x24,0x01,0x24,0x01,0x98,0x00,0x00},/  
*"s",83*/  
  
{0x00,0x00,0x01,0x00,0x01,0x00,0x07,0xF8,0x01,0x04,0x01,0x04,0x00,0x00,0x00,0x00},/  
*"t",84*/  
  
{0x01,0x00,0x01,0xF8,0x00,0x04,0x00,0x04,0x00,0x04,0x01,0x08,0x01,0xFC,0x00,0x04},/  
*"u",85*/  
  
{0x01,0x00,0x01,0x80,0x01,0x70,0x00,0x0C,0x00,0x10,0x01,0x60,0x01,0x80,0x01,0x00},/  
*"v",86*/  
  
{0x01,0xF0,0x01,0x0C,0x00,0x30,0x01,0xC0,0x00,0x30,0x01,0x0C,0x01,0xF0,0x01,0x00},/  
*"w",87*/  
  
{0x00,0x00,0x01,0x04,0x01,0x8C,0x00,0x74,0x01,0x70,0x01,0x8C,0x01,0x04,0x00,0x00},/  
*"x",88*/  
  
{0x01,0x01,0x01,0x81,0x01,0x71,0x00,0x0E,0x00,0x18,0x01,0x60,0x01,0x80,0x01,0x00},/  
*"y",89*/  
  
{0x00,0x00,0x01,0x84,0x01,0x0C,0x01,0x34,0x01,0x44,0x01,0x84,0x01,0x0C,0x00,0x00},/  
*"z",90*/  
  
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x00,0x3E,0xFC,0x40,0x02,0x40,0x02},/  
*"{",91*/  
  
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xFF,0xFF,0x00,0x00,0x00,0x00,0x00,0x00},/  
*"|",92*/  
  
{0x00,0x00,0x40,0x02,0x40,0x02,0x3E,0xFC,0x01,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/  
*"}",93*/  
  
{0x00,0x00,0x60,0x00,0x80,0x00,0x80,0x00,0x40,0x00,0x40,0x00,0x20,0x00,0x20,0x00},/  
*"~",94*/  
};  
  
const unsigned char oled_CH_1616[][12]={  
/////////////////////////////////////  
{0x00,0x00,0x4F,0xF0,0x28,0x00,0x0B,0xC0,0x0A,0x40,0xFA,0x40},  
{0x0A,0x40,0x0B,0xC0,0x28,0x10,0x4F,0xF0,0x00,0x00,0x00,0x00},/*"尚",0*/  
  
{0x42,0x00,0x4F,0xE0,0x74,0x40,0x47,0xE0,0x00,0x10,0x24,0x90},
```

```
{0x24,0x90,0xFD,0xF0,0x24,0x90,0x24,0x90,0x04,0x10,0x00,0x00},/*"硅",1*/

{0x02,0x00,0x22,0x00,0x45,0xF0,0x89,0x20,0x11,0x20,0x21,0x20},
{0x11,0x20,0x89,0x20,0x45,0xF0,0x22,0x00,0x02,0x00,0x00,0x00},/*"谷",2*/

{0x04,0x00,0x6F,0xF0,0x24,0x90,0x24,0x90,0x2F,0xF0,0xE4,0x00},
{0x22,0x10,0x3C,0x60,0x25,0x80,0x64,0x60,0x06,0x10,0x00,0x00},/*"嵌",3*/

{0x00,0x10,0x00,0x20,0x80,0x40,0x81,0x80,0x4E,0x00,0x30,0x00},
{0x0E,0x00,0x01,0x80,0x00,0x40,0x00,0x20,0x00,0x10,0x00,0x00},/*"入",4*/

{0x10,0x10,0x12,0x10,0x12,0x10,0x13,0xE0,0x12,0x20,0x12,0x20},
{0x10,0x20,0xFF,0x00,0x10,0xC0,0x90,0x20,0x50,0x70,0x00,0x00},/*"式",5*/

//////////////////////////////////////
//信号图标

{0x00,0x00,0x00,0x00,0x00,0x00,0x10,0x80,0x09,0x00,0x06,0x00},
{0x06,0x00,0x09,0x00,0x10,0x80,0x00,0x00,0x00,0x00,0x00,0x00},/*"x",0*/

{0x00,0x00,0x00,0x30,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/*"一竖",1*/

{0x00,0x00,0x00,0x30,0x00,0x00,0x00,0xF0,0x00,0x00,0x00,0x00},
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/*"两竖",2*/

{0x00,0x00,0x00,0x30,0x00,0x00,0x00,0xF0,0x00,0x00,0x03,0xF0},
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/*"三竖",3*/

{0x00,0x00,0x00,0x30,0x00,0x00,0x00,0xF0,0x00,0x00,0x03,0xF0},
{0x00,0x00,0x0F,0xF0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/*"四竖",4*/

{0x00,0x00,0x00,0x30,0x00,0x00,0x00,0xF0,0x00,0x00,0x03,0xF0},
{0x00,0x00,0x0F,0xF0,0x00,0x00,0x3F,0xF0,0x00,0x00,0x00,0x00},/*"五竖",5*/

//////////////////////////////////////
```

```
//进度条
```

```
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
```

```
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},//6
```

```
{0x7F,0xE0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
```

```
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},//7
```

```
{0x7F,0xE0,0x7F,0xE0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
```

```
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},//8
```

```
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x00,0x00,0x00,0x00,0x00,0x00},
```

```
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},//9
```

```
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x00,0x00,0x00,0x00},
```

```
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},//10
```

```
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x00,0x00},
```

```
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},//11
```

```
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0},
```

```
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},//12
```

```
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0},
```

```
{0x7F,0xE0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},//13
```

```
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0},
```

```
{0x7F,0xE0,0x7F,0xE0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},//14
```

```
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0},
```

```
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x00,0x00,0x00,0x00,0x00,0x00},//15
```

```
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0},
```

```
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x00,0x00,0x00,0x00},//16
```

```

{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0},
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x00,0x00},//17

{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0},
{0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0,0x7F,0xE0},//18
////////////////////////////////////
//电灯图标

{0x00,0x00,0x00,0x00,0x4C,0x00,0x2A,0x00,0x09,0xE0,0x28,0x20},
{0x28,0x20,0x09,0xE0,0x2A,0x00,0x4C,0x00,0x00,0x00,0x00,0x00},/*"开",19*/

{0x00,0x00,0x40,0x20,0x26,0x40,0x15,0x80,0x0D,0xE0,0x16,0x20},
{0x16,0x20,0x0D,0xE0,0x15,0x80,0x26,0x40,0x40,0x20,0x00,0x00},/*"关",20*/

};

const unsigned char oled_CH15[][12]={
{0x21,0x00,0xA2,0x00,0x6F,0xF0,0x34,0x00,0x02,0x10,0x7F,0x20},
{0x40,0xC0,0x5F,0x00,0x40,0xE0,0x7F,0x10,0x00,0x30,0x00,0x00},/*"?",0*/

{0x38,0x10,0xA0,0x10,0x6F,0xA0,0x28,0x20,0xA8,0x40,0x6B,0x80},
{0x28,0x60,0x28,0x10,0x6F,0x90,0xA0,0x10,0x38,0x30,0x00,0x00},/*"?",1*/

{0x34,0x10,0x24,0x20,0x25,0xC0,0x24,0x20,0xA4,0x10,0x67,0xF0},
{0x24,0x90,0x24,0x90,0x24,0x90,0x24,0x90,0x34,0x10,0x00,0x00},/*"?",2*/

{0x08,0x00,0x3F,0xF0,0xC0,0x00,0x10,0x10,0x16,0x10,0x11,0xD0},
{0x90,0x10,0x50,0x30,0x10,0xD0,0x17,0x10,0x10,0x10,0x00,0x00},/*"?",3*/

```

```
{0x11,0x00,0x16,0x00,0xFF,0xF0,0x12,0x00,0x40,0x50,0x5F,0x50},
{0xF5,0x60,0x55,0xC0,0xF5,0x60,0x5F,0x50,0x40,0x50,0x00,0x00},/*"?",4*/

{0x10,0x40,0xFF,0xC0,0x10,0x80,0x02,0x10,0x22,0x20,0x22,0xC0},
{0xFF,0x00,0x22,0xC0,0x22,0x20,0x3E,0x10,0x02,0x10,0x00,0x00},/*"?",5*/

{0x00,0x00,0x00,0x00,0x0C,0xC0,0x0C,0xC0,0x00,0x00,0x00,0x00},
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},/*":",6*/
};
```

7.2 应用层

7.2.1 App_Display.h

```
#ifndef __APP_DISPLAY_H
#define __APP_DISPLAY_H

#include "Common_Debug.h"
#include "oled.h"
#include "oledfont.h"
#include "Inf_Si24R1.h"
#include "Inf_JoyStickKey.h"

#define Line1_Begin 29
#define Line2_Begin 5
#define Line3_Begin 5
#define Line4_Begin 30
#define Line5_Begin 2

#define X_Begin 0
#define Y_Begin 51
#define Z_Begin 103

#define Line1_Begin1 0
#define Line2_Begin1 0
#define Line3_Begin1 40
#define Line4_Begin1 0
```

```
#define Line5_Begin1 0

#define Y0 0
#define Y1 14
#define Y2 Y1 + 12
#define Y3 Y2 + 12
#define Y4 Y3 + 12
#define Y5 Y4 + 12

typedef struct
{
    uint8_t  hardware_type;
    uint8_t  oled_delay;
    uint8_t  windows;
    uint8_t  low_power;
    uint8_t  NRF_Err;
    uint8_t  Rc_num;
    uint8_t  Connect_Succeed;
    uint16_t Battery_Rc;
    uint16_t Battery_Fly;
    uint16_t test_flag, set_flag;
    int      X, Y, Z, H;
} Show_Struct;

extern Show_Struct Show;
extern uint16_t      set_temp;
extern uint16_t      test_temp;
extern uint8_t       send_flag;

void App_Display_Start(void);

void App_Display_Show(void);
#endif
```

7.2.2 App_Display.c

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
#include "App_Display.h"

void App_Display_ShowProgressBar(uint8_t temp,
                                uint8_t chr_star,
                                uint8_t chr_default,
                                uint8_t x,
                                uint8_t y,
                                uint8_t size,
                                uint8_t mode);

void App_Display_Start(void)
{
    OLED_Init();
}

void App_Display_Show(void)
{
    uint8_t temp;

    ////////////////////////////////////第一行//////////////////////////////////////

    OLED_Show_CH(Line1_Begin + 00, Y0, 0, 12, 1);
    OLED_Show_CH(Line1_Begin + 12, Y0, 1, 12, 1);
    OLED_Show_CH(Line1_Begin + 24, Y0, 2, 12, 1);
    OLED_Show_CH(Line1_Begin + 36, Y0, 3, 12, 1);
    OLED_Show_CH(Line1_Begin + 48, Y0, 4, 12, 1);
    OLED_Show_CH(Line1_Begin + 60, Y0, 5, 12, 1);

    OLED_ShowNumber(2, Y0, CH, 3, 12);    // 显示无线信道

    ////////////////////////////////////第二行//////////////////////////////////////

    ////////////////////////////////////第三、四行//////////////////////////////////////
    // 显示遥控数据

    OLED_ShowString(Line3_Begin + 00, Y2, "THR:", 12, 1);

    temp = joyStick.THR / 41;

    App_Display_ShowProgressBar(temp, 12, 24, Line4_Begin + 6, Y2, 12, 1);

    temp = (joyStick.THR - 500) / 41;
```

```
App_Display_ShowProgressBar(temp, 12, 12, Line4_Begin + 18, Y2, 12, 1);

OLED_ShowString(Line3_Begin + 64, Y2, "ROL:", 12, 1);
temp = joyStick.ROL / 41;
App_Display_ShowProgressBar(temp, 12, 24, Line4_Begin + 70, Y2, 12, 1);
temp = (joyStick.ROL - 500) / 41;
App_Display_ShowProgressBar(temp, 12, 12, Line4_Begin + 82, Y2, 12, 1);

OLED_ShowString(Line3_Begin + 00, Y3, "YAW:", 12, 1);
temp = joyStick.YAW / 41;
App_Display_ShowProgressBar(temp, 12, 24, Line4_Begin + 6, Y3, 12, 1);
temp = (joyStick.YAW - 500) / 41;
App_Display_ShowProgressBar(temp, 12, 12, Line4_Begin + 18, Y3, 12, 1);

OLED_ShowString(Line3_Begin + 64, Y3, "PIT:", 12, 1);
temp = joyStick.PIT / 41;
App_Display_ShowProgressBar(temp, 12, 24, Line4_Begin + 70, Y3, 12, 1);
temp = (joyStick.PIT - 500) / 41;
App_Display_ShowProgressBar(temp, 12, 12, Line4_Begin + 82, Y3, 12, 1);

OLED_Refresh_Gram();    // 开始显示
}

// 进度条显示函数
void App_Display_ShowProgressBar(uint8_t temp,
                                uint8_t chr_star,
                                uint8_t chr_default,
                                uint8_t x,
                                uint8_t y,
                                uint8_t size,
                                uint8_t mode)
{
    switch(temp)
    {

```

```
    case 0: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 1: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 2: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 3: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 4: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 5: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 6: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 7: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 8: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 9: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 10: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 11: OLED_Show_CH(x, y, chr_star + temp, size, size); break;
    case 12: OLED_Show_CH(x, y, chr_star + temp, size, size); break;

    default: OLED_Show_CH(x, y, chr_default, size, size); break;
}
}
```

7.3 任务调度

```
/* 4. 显示任务 */
void          displayTask(void *arg);
TaskHandle_t displayHandle;
#define DISPLAY_PRIORITY 8
#define DISPLAY_EXECUTE_CYCLE 10
/* 4. 显示任务 */
void displayTask(void *arg)
{
    debug_printfln("显示任务：开始调度");
    TickType_t pxPreviousWakeTime = xTaskGetTickCount();
    while(1)
    {
        App_Display_Show();
        xTaskDelayUntil(&pxPreviousWakeTime, JOYSTICK_EXECUTE_CYCLE);
    }
}
```

```
}
```